

Issue 1: Database Download Failures

Copy

flowchart TD

```
A[DB Download Fails] --> B{Error Type?}
B -->|Timeout| C[Increase Timeout]
B -->|Auth Error| D[Configure Credentials]
B -->|Network Error| E[Check Proxy Settings]
B -->|Rate Limited| F[Use Token or Mirror]
```

```
C --> G[trivy --timeout 30m]
```

```
D --> H[Set GITHUB_TOKEN]
```

```
E --> I[Set HTTP_PROXY]
```

```
F --> J[Configure Registry]
```

Symptoms:

Copy

FATAL failed to download vulnerability DB

Solutions:

Copy

1. Increase timeout for slow connections

```
trivy image --timeout 30m nginx:latest
```

2. Configure proxy for corporate networks

```
export HTTP_PROXY=http://proxy.company.com:8080
```

```
export HTTPS_PROXY=http://proxy.company.com:8080
```

```
export NO_PROXY=localhost,127.0.0.1
```

```
trivy image nginx:latest
```

3. Authenticate to avoid rate limiting

```
export GITHUB_TOKEN=your_github_token
```

```
trivy image nginx:latest
```

4. Use a mirror registry

```
trivy image --db-repository your-registry.com/trivy-db nginx:latest
```

5. Clear corrupted cache and retry

```
rm -rf ~/.cache/trivy/db/
```

```
trivy image nginx:latest
```

Issue 2: Image Pull Failures

Symptoms:

Copy

FATAL failed to pull image

Solutions:

Copy

1. Check image exists

```
docker pull nginx:latest
```

2. Authenticate to private registry

```
export TRIVY_USERNAME=your_username
```

```
export TRIVY_PASSWORD=your_password
```

```
trivy image private-registry.com/myapp:latest
```

3. Use Docker credentials

Trivy reads from ~/.docker/config.json

```
docker login private-registry.com
```

```
trivy image private-registry.com/myapp:latest
```

4. Scan local image instead

`docker pull nginx:latest`

`trivy image --input nginx.tar nginx:latest`

5. Skip TLS verification (development only)

`trivy image --insecure private-registry.com/myapp:latest`

Issue 3: Timeout During Scan

Symptoms:

Copy

FATAL context deadline exceeded

Solutions:

Copy

1. Increase overall timeout

`trivy image --timeout 60m large-image:latest`

2. Skip database update (use cached)

`trivy image --skip-db-update nginx:latest`

3. Scan specific layers only

`trivy image --ignore-unfixed nginx:latest`

4. Reduce scan scope

`trivy image --vuln-type os nginx:latest` *# OS packages only*

`trivy image --vuln-type library nginx:latest` *# Libraries only*

5. Use parallel scanning for multiple images

Pre-download DB once

```
trivy image --download-db-only
```

Then scan without DB update

```
for img in image1 image2 image3; do
```

```
    trivy image --skip-db-update $img &
```

```
done
```

```
wait
```

Issue 4: No Vulnerabilities Detected

Symptoms:

Copy

Total: 0 (UNKNOWN: 0, LOW: 0, MEDIUM: 0, HIGH: 0, CRITICAL: 0)

Possible Causes and Solutions:

Copy

1. Unsupported OS - check what Trivy detects

```
trivy image --debug nginx:latest 2>&1 | grep "Detected OS"
```

2. Outdated database - force refresh

```
trivy image --reset nginx:latest
```

3. Image uses scratch base - no OS packages

This is expected for distroless images

```
trivy image --vuln-type library gcr.io/distroless/static:latest
```

4. Check if files are detected

```
trivy image --debug nginx:latest 2>&1 | grep -i "analyzing"
```

5. Scan filesystem instead of image

```
docker create --name temp nginx:latest
```

```
docker export temp | tar -xf - -C ./extracted/
```

```
docker rm temp
```

```
trivy fs ./extracted/
```

Issue 5: False Positives

Symptoms: Trivy reports vulnerabilities that do not actually affect your application.

Solutions:

Copy

1. Check vulnerability details

```
trivy image --format json nginx:latest | \
```

```
jq '.Results[].Vulnerabilities[] | select(.VulnerabilityID == "CVE-2023-12345")'
```

2. Verify the vulnerable function is used

Check if the vulnerable code path exists in your application

3. Create ignore file for confirmed false positives

```
cat > .trivyignore << EOF
```

```
# False positive: function not used in our code
```

```
CVE-2023-12345
```

```
# Mitigated by network policy
```

```
CVE-2023-67890
```

```
EOF
```

```
trivy image --ignorefile .trivyignore nginx:latest
```

4. Ignore with expiration date

```
echo "exp:2024-06-01 CVE-2023-11111" >> .trivyignore
```

5. Filter by status (ignore unfixed)

```
trivy image --ignore-unfixed nginx:latest
```

Issue 6: Missing Vulnerabilities

Symptoms: Known CVEs are not being reported.

Solutions:

Copy

1. Check database freshness

```
trivy image --debug nginx:latest 2>&1 | grep -i "database"
```

Check metadata

```
cat ~/.cache/trivy/db/metadata.json
```

2. Force database update

```
trivy image --reset nginx:latest
```

3. Check if CVE is in supported database

Trivy may not track all vulnerability sources

4. Check severity filter

```
trivy image --severity CRITICAL,HIGH,MEDIUM,LOW,UNKNOWN nginx:latest
```

5. Verify package version detection

```
trivy image --format json nginx:latest | \
```

```
jq '.Results[].Packages[] | select(.Name == "openssl")'
```

Trivy Not Found

Symptoms:

- Error message indicating Trivy executable not found
- Scans fail to start

Solutions:

1. Verify that Trivy is installed on your system
2. Check the Trivy path in Settings → Tools → Trivy: Settings
3. Make sure the path points to the correct Trivy executable
4. If using just trivy as the path, ensure Trivy is in your system PATH

Scan Results Not Showing

Symptoms:

- Scan completes but no results are displayed
- "No vulnerabilities found" message when you expected findings

Solutions:

1. Check if severity filters in settings are too restrictive
2. Verify that the appropriate scanners are enabled (vulnerability, misconfiguration, secret)
3. Make sure the "Only show issues with fixes" option isn't filtering out your findings
4. Try running Trivy directly from the command line to verify it finds issues

Cannot Open Files from Results

Symptoms:

- Clicking on findings doesn't open the corresponding file
- Error message when trying to navigate to a file

Solutions:

1. Make sure the file paths are correct and accessible
2. Check if the file is part of the current project
3. Verify that the file hasn't been moved or renamed since the scan
4. Try rerunning the scan

Slow Scan Performance

Symptoms:

- Scans take an unusually long time to complete

Solutions:

1. Consider using the offline scan option if you're primarily concerned with known vulnerabilities
2. Limit the scan to only necessary severity levels
3. For large projects, consider scanning specific directories or files rather than the entire project
4. Ensure your Trivy database is up to date (trivy --download-db-only)

Issues with Aqua Platform Integration

Symptoms:

- Cannot connect to Aqua Platform
- Assurance policy results not showing

Solutions:

1. Verify your API Key and API Secret are entered correctly
2. Ensure you have selected the correct region
3. Check your network connectivity to the Aqua Platform
4. Verify your Aqua Platform subscription is active

Une étude approfondie des discussions GitHub de **Trivy** révèle plusieurs erreurs fréquemment rapportées :

- **Détection manquante de vulnérabilités** (faux négatifs) : par exemple, Trivy ne détecte pas la CVE-2025-55130 dans des images Node.js construites hors gestionnaire de paquets. Les mainteneurs confirment que c'est un comportement « par conception » (Trivy ne scanne que les paquets OS/gestionnaire, pas les binaires compilés) et conseillent d'installer Node via un gestionnaire de paquets pour corriger le problème.
- **Problèmes d'installation / dépôt APT** : un utilisateur rapportait une erreur « jammy InRelease is not signed » lors du apt update, due au fait que la version demandée (0.69.3) n'était pas dans le dépôt APT mis à jour et que la clé publique

avait changé. La solution proposée a été d'utiliser directement les paquets GitHub Releases correspondants ou d'attendre la mise à jour du dépôt par les mainteneurs.

- **Artefacts manquants / comptes compromis** : plusieurs discussions concernent l'absence de tags latest sur Docker Hub ou d'artefacts pour certaines versions (0.62.1, etc.) après un incident de sécurité. Par exemple, le tag latest sur Docker Hub n'est plus disponible car le compte a été « verrouillé » pour sécurité. Les mainteneurs recommandent d'utiliser le registre GHCR/ECR ou de pinner une version précise. De même, les artefacts Windows/linux manquants pour v0.62.1 sont en cours de restauration et liés à l'incident de sécurité.
- **Erreurs lors de la génération de SBOM** : un bug est rapporté lors de la génération d'un SBOM pour une image VMDK VMware ; Trivy échoue à parser le descripteur de disque en raison d'espaces superflus (erreur « failed to parse version »). Le rapport souligne que le format VMDK autorise ces espaces. La correction impliquerait de mettre à jour la bibliothèque de lecture VMDK ou de nettoyer les espaces, mais le thread reste ouvert.
- **Divergence d'analyse VEX entre image et SBOM** : un utilisateur signale qu'un fichier VEX (exclusion de vulnérabilités) est appliqué lors d'un scan d'image Docker mais ignoré lors du scan d'un SBOM généré. Ce comportement inattendu (différence de détection entre trivy image et trivy sbom) reste non résolu.
- **Détection de vulnérabilités JavaScript** : une confusion sur le scan d'images Node.js montre que les fichiers package-lock.json/yarn.lock ne sont pas pris en compte dans les analyses d'image (seulement dans fs). Les mainteneurs précisent que l'analyse d'image ne gère pas ces fichiers lock pour le moment.

Recommandations clés : Mettez à jour Trivy vers la dernière version (qui corrige de nombreuses vulnérabilités embarquées), nettoyez les espaces dans les fichiers VMDK ou attendez une mise à jour de la librairie, et pour l'installation préférez les paquets officiels (GitHub Releases). Pour les images, utilisez de préférence GHCR ou ECR et pinner les versions au lieu de :latest. La figure ci-dessous illustre les liens entre les discussions et les autres ressources (issues, commits) pour les principaux incidents.

Tableau des erreurs

ID	Titre	Résumé de l'erreur	Corrections proposées	Versions affectées	Reproductions
1	CVE-2025-55130 non détectée par Trivy	Trivy n'a pas détecté la CVE-2025-55130 dans des images Node.js (vuln. présente).	Pas de correctif : par conception Trivy ne scanne que les paquets installés via gestionnaire. Utiliser images Node via apt/yum ou scruter avec npm audit.	0.70.0 (action GH)	Scan d'une image Node.js Alpine via trivy-action.
2	Erreur de signature dépôt APT Trivy	apt-get update échoue (« jammy InRelease is not signed »). Le dépôt APT Trivy n'a pas la v0.69.3 et la clé publique a changé.	Utiliser le paquet .deb depuis GitHub Releases (tag v0.69.3) ou attendre la mise à jour du dépôt. La clé publique a été mise à jour (#10549).	TRIVY_VERSION=0.69.3	Dockerfile construisant Trivy 0.69.3 via le repo APT.
3	Tag Docker Hub « latest » manquant	Le tag aquasec/trivy:latest n'existe plus : Docker Hub a bloqué le dépôt après compromission.	Attendre déblocage ou utiliser d'autres registres (GHCR/ECR), ou mieux <i>pinner</i> une version précise plutôt que latest.	—	docker pull aquasec/trivy:latest échoue.
4	Différence de résultat VEX SBOM vs image	Un fichier VEX exclut des vulnérabilités pour l'image, mais est ignoré lors du scan du SBOM généré (vuln. non exclue).	Non résolu. (Comportement inattendu ; Trivy devrait appliquer les mêmes exclusions au SBOM). Peut nécessiter un correctif du moteur VEX pour SBOM.	0.69.3	Comparaison entre trivy image --vex ... et trivy sbom --vex
5	Échec publication	Depuis le 19 mars, la mise à jour quotidienne de la base Java-	Pas de correction immédiate. Les mainteneurs restaurent les	v0.67.2	Automatique (GitHub Actions journalières depuis 2026-03-19).

ID	Titre	Résumé de l'erreur	Corrections proposées	Versions affectées	Reproductions
	n Java-DB Trivy	DB Trivy a échoué (GitHub Actions « bad credentials »).	pipelines après un incident de sécurité.		
6	Artefacts manquants pour v0.62.1	Les archives (Windows/Linux/etc.) pour la version Trivy v0.62.1 sont absentes (probablement suite à l'incident).	Correctif en cours. Les mainteneurs ont lié cela à l'incident de sécurité (#10425) et restaureront les artefacts.	—	—
7	SBOM VMDK échoue (« failed to parse disk descriptor »)	trivy vm --format=... fichier.vmdk échoue (« failed to parse version: 1 ») car le fichier descripteur VMDK contient des espaces en fin de ligne.	Nettoyer le descripteur (supprimer espaces) ou mettre à jour la librairie de parsing VMDK pour tolérer ces espaces (bug en suspens).	0.69.3 (immédiat)	trivy vm --format=spdx ubuntu-...-disk-0.vmdk retourne l'erreur ci-dessus.
8	Détection de vulnérabilités Node.js différente (fs vs image)	Un scan trivy fs détecte la vulnérabilité, mais trivy image (sans node_modules) ne la détecte pas (le package.json de l'app n'est pas pris en compte).	Conçu ainsi : le scan image ne gère pas les lockfiles JS. Les mainteneurs confirment que package.json seul n'est pas suffisant, et n'ont pas de plan de corriger le scan image pour supporter package-lock.json.	0.69.3	Comparaison de trivy fs . vs trivy image ... sur un projet Node.js.
9	Inclusion de vulnérabilité Go dans	Trivy 0.69.3 embarque une version vulnérable de Go (stdlib	Mettre à jour la version de Go ou Trivy à 0.69.4+ qui contient la correction (Go 1.26) pour supprimer la CVE.	0.69.3	Scanner une image contenant Trivy v0.69.3 via trivy image.

ID	Titre	Résumé de l'erreur	Corrections proposées	Versions affectées	Reproductions
	Trivy 0.69.3	v1.25.7) avec la CVE-2026-25679, provoquant une erreur haute lors du scan d'images qui installent Trivy.			
10	Scan de python:3.12.13-slim echo (entête TAR invalide)	Trivy v0.69.2 plantait sur l'image python:3.12.13-slim avec une erreur TAR invalid (plusieurs commentaires dans le thread).	Correctif déployé dans Trivy v0.70.0 (le problème venait d'une mauvaise lecture des entêtes TAR).	0.69.2	trivy image python:3.12.13-slim dans le CI.